

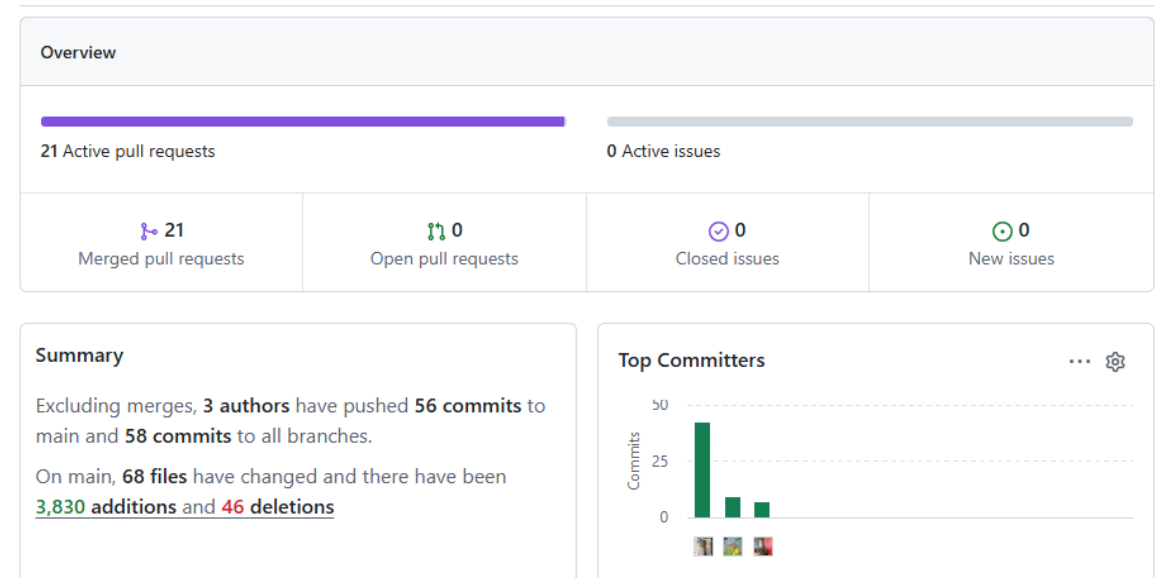


KOBO BUSINESS MERCHANT PROJECT

- ▶ Presented by:
Khumbelo Thandavhathu
- ▶ Jason Barron
- ▶ Khangweni Mulaudzi

Team Workflow and Git Contributions

- ▶ Development followed a Git-based workflow using feature branches
- ▶ Changes were merged into the main branch via pull requests
- ▶ A total of 21 pull requests were successfully merged



Technical Demo: Automated Execution of a complex Merchant journey

Objective

- Validate end-to-end merchant workflow using automation
- Ensure system reliability across multiple user actions

Scenario Overview

- Merchant logs into the platform
- Browses and selects multiple products
- Updates product quantities dynamically
- Proceeds through checkout process
- Generates and verifies transaction receipt

Technical Demo: Playwright test script

```
3 test('test', async ({ page }) => {  
4  
5   // Navigate to the login page  
6   await page.goto('https://quality-engineering-labs.vercel.app/login.html');  
7  
8   // Enter merchant username  
9   await page.getByTestId('login-username').click();  
10  await page.getByTestId('login-username').fill('admin');  
11  
12  // Enter merchant password  
13  await page.getByTestId('login-password').click();  
14  await page.getByTestId('login-password').fill('password123');  
15  
16  // Submit login form  
17  await page.getByTestId('login-submit').click();  
18  
19  // Navigate to the Products page via main navigation  
20  await page.getByLabel('Main navigation').getByRole('link', { name: 'Products' }).click();  
21  
22  // Add first product to cart  
23  await page.getByTestId('add-cart-1').click();  
24  
25  // Add second product to cart  
26  await page.getByTestId('add-cart-2').click();  
27  
28  // Increase quantity of second product (testing quantity update functionality)  
29  await page.getByTestId('qty-plus-2').click();  
30  
31  // Open the shopping cart  
32  await page.getByTestId('fab-cart').click();  
33  
34  // Proceed to checkout  
35  await page.getByTestId('checkout-btn').click();  
36  
37  // Verify that checkout items are displayed (ensures cart data is carried forward correctly)  
38  await expect(page.getByTestId('checkout-items')).toBeVisible();  
39  
40  // Confirm the checkout process  
41  await page.getByTestId('confirm-checkout-btn').click();  
42  
43  // Continue shopping after successful checkout  
44  await page.getByTestId('continue-shopping-btn').click();  
45  
46  // Navigate to the Receipts page  
47  await page.getByLabel('Main navigation').getByRole('link', { name: 'Receipts' }).click();  
48  
49  // Open the most recent receipt (index 0 assumed latest)  
50  await page.getByTestId('view-receipt-0').click();  
51  
52  // Verify that the receipt page is displayed correctly  
53  await expect(page.getByRole('heading', { name: 'MerchantHub Receipt' })).toBeVisible();  
54  });
```

Technical Demo: Playwright script technical analysis

Technical Execution

- Built using Playwright automation framework
- Utilised data-testid selectors for stability
- Simulated real user interactions (clicks, inputs, navigation)
- Assertions used to validate:
 - Checkout items visibility
 - Successful transaction completion
 - Receipt generation

Technical Demo: Playwright merchant journey

The screenshot displays the Playwright Test interface, which is used for automating web browser interactions. The interface is divided into several sections:

- Top Panel:** A timeline showing the execution of the test, with a progress bar and a 'Pause' button.
- Left Panel:** A list of actions performed during the test, including navigation, clicking, and filling form fields. The actions are categorized by time and status (e.g., 'Passed', 'Failed').
- Center Panel:** A screenshot of the web application being tested, showing the 'Merchant Sign In' page. The page includes a login form with fields for 'Username' and 'Password', a 'Remember me' checkbox, and a 'Sign In' button. The URL in the browser is `https://quality-engineering-labs.vercel.app/login.html`.
- Bottom Panel:** A code editor showing the test script written in JavaScript using Playwright's API. The script includes imports, test definitions, and specific actions like `page.goto`, `page.getByTestId`, `page.fill`, and `page.click`.

The test script in the bottom panel is as follows:

```
1 import { test, expect } from '@playwright/test';
2
3 test('test', async ({ page }) => {
4   await page.goto('https://quality-engineering-labs.vercel.app/login.html');
5   await page.getByTestId('login-username').click();
6   await page.getByTestId('login-username').fill('admin');
7   await page.getByTestId('login-password').click();
8   await page.getByTestId('login-password').fill('password123');
9   await page.getByTestId('login-submit').click();
10  await page.getByLabel('Main navigation').getByRole('link', { name: 'Products' }).click();
11 }
```


Risk Analysis: Identification of the most dangerous financial or security vulnerability discovered.

Financial Risks

Incorrect Commission Calculation

Float data type for financial values

No transaction handling

Security Risks

Session Timeout Failure

No Role-Based Access Control(RBAC)



Incorrect Commission Calculation

The system calculates commission values incorrectly, resulting in mismatches between expected and stored values.

Business Impact:

This can lead to revenue loss or overcharging customers, inaccurate financial reporting, and loss of trust from customers.



Float data type for financial values

Using float for financial calculations introduces rounding errors due to accuracy limitations.

Business Impact:

Small rounding errors can build up over many transactions, leading to significant financial differences over time.



No Transaction Handling

Transactions are not wrapped in a database transaction, meaning steps like wallet deduction and voucher issuance are not executed as a single unit.

Business Impact:

This can result in partial failures, such as money being deducted without issuing a voucher, leading to financial loss and customer complaints.

Session Timeout Failure

Users remain logged in even after session expiry or cookie clearing.

Business Impact:

This allows unauthorized access to accounts, increasing the risk of fraud, data breaches, and financial loss.

No Role-Based Access Control(RBAC)

Merchants can access sensitive data such as settlements due to missing access restrictions.

Business Impact:

This exposes confidential financial data, violates data protection regulations (POPIA), and can severely damage business reputation and trust.



Data Reconciliation: SQL evidence demonstrating that the ledger state matches the transactional logic of the API.

Key Reconciliation Checks:

Ledger vs Voucher Consistency

Every successful transaction should have a voucher.

Finding:

- “Ghost Transactions” exist → wallet debited but no voucher issued
- Indicates API failure or partial processing

Duplicate Transaction Detection (Idempotency Failure)

This checks if the same transaction reference appears more than once.

Finding:

- Duplicate ExternalReference values detected
- Leads to **duplicate charges for a single request**

Settlement Integrity Check

Net = Gross – Commission

If it doesn't match, it means financial reports are incorrect.

Finding:

- Settlement mismatches found
- Financial reports are **inaccurate**

Commission Accuracy

This checks if there are multiple commission rules for the same product and tier.

Finding:

- Duplicate commission rules exist
- Causes **incorrect commission calculations**

Strategic Answers to the Task Questions

Task 1 Question

Based on the principles of traceability and accountability, how does a structured Git branching strategy reduce the risk of regression when three engineers are contributing to a shared automation suite?

Task 2 Question

Task 2 Question: During API validation, did you identify any instances where the system failed to enforce "Least Privilege" access, potentially allowing a Merchant to view or modify data belonging to another Merchant?

Task 3 Question

Task 3 Question: In the context of the Page Object Model (POM), how does the separation of "Locators" from "Test Logic" improve the stability of your automation suite when the UI undergoes structural changes?

Task 4 Question

Task 4 Question: Identify a specific data type used in the financial tables that could cause rounding errors over high-volume transactions. Provide a SQL example demonstrating a potential financial mismatch.

```
-- Transaction Engine
CREATE TABLE TransactionLedger (
    EntryID UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    WalletID INT FOREIGN KEY REFERENCES Wallets(WalletID),
    ProductID INT FOREIGN KEY REFERENCES Products(ProductID),
    Amount FLOAT NOT NULL,
    ExternalReference NVARCHAR(100),
    ProcessingStatus NVARCHAR(20),
    CreatedTimestamp DATETIME DEFAULT GETDATE()
);
```


TASK 4 EVIDENCE

```
11 select CommissionAmount
12 from TransactionLedger
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Results (1) Messages

	CommissionAmount	↕	🔍
1	1,3499999999999999		
2	0,5		
3	1		
4	3		
5	0,6749999999999999		
6	0,25		
7	3		
8	1		
9	1		
10	1		
11	1,5		
12	1,65		
13	1		
14	3		
15	3		
16	2,75		